



Università degli Studi di Messina

Corso di laurea in Ingegneria Elettronica ed Informatica

A.A. 2021/22

Fondamenti di Informatica

Verso il progetto

Prof. Francesco Longo

Composizione di strutture dati

Composizione di strutture dati

- Così come i costrutti di base del C (tipi di dato, array, struct, puntatori, ...) possono essere composti per creare strutture dati dinamiche e contenitori così quest'ultime possono essere composte tra loro per creare strutture dati ancora più complesse.
- All'interno di software evoluti è molto comune, ad esempio, avere la necessità di gestire:
 - liste di liste;
 - liste di pile;
 - liste di code;
 - ...
- Gli esempi di composizioni che vedremo saranno:
 - utilizzo di una struct come contenuto informativo di una lista;
 - liste di liste.

Struct come contenuto informativo

- Negli esempi trattati nei blocchi precedenti abbiamo utilizzato come contenuto informativo un intero per rendere più semplici le implementazioni ma in realtà ogni nodo della lista potrebbe essere caratterizzato da più campi informativi i quali possono essere a loro volta raggruppati in una struttura.
- Immaginiamo di voler implementare una lista di punti nel piano (che rappresenta una spezzata).

Essa può essere implementata come:

```
struct                polyline{
    float              x;
    float              y;
    struct             polyline* next;
};
```

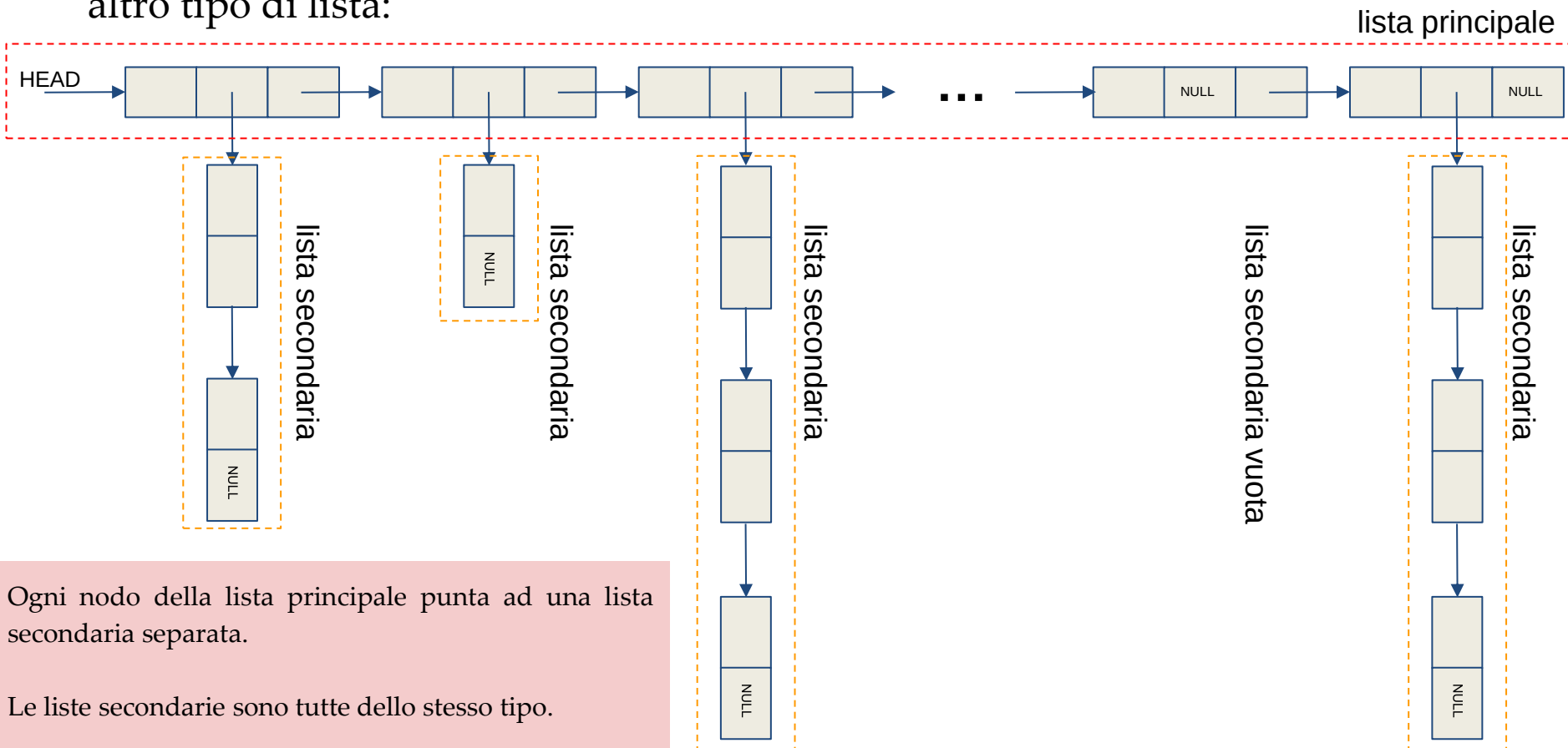
Ma ha molto più senso qualcosa di questo genere:

```
struct point{
    float x;
    float y;
};

struct polyline{
    struct point p;
    struct polyline* next;
};
```

Lista di liste

- Ogni nodo della lista potrebbe contenere a sua volta il puntatore alla testa di un altro tipo di lista:



Ogni nodo della lista principale punta ad una lista secondaria separata.

Le liste secondarie sono tutte dello stesso tipo.

Le lunghezze delle varie liste secondarie non sono necessariamente legate tra loro.

Lista di liste (2)

Volendo ad esempio implementare una lista di spezzate ad ognuna delle quali è assegnato un nome potremmo scrivere in C:

```
struct point{  
    float x;  
    float y;  
};
```

```
struct polyline{  
    struct point p;  
    struct polyline* next;  
};
```

Contenuto informativo della lista secondaria

```
struct polylines{  
    char name[20];  
    struct polyline* pl;  
    struct polylines* next;  
}
```

Contenuto informativo della lista principale

Puntatore alla testa della lista secondaria

Uso di ADT

- Una struttura dati diventa un ADT quando viene associata alla sua interfaccia e qualunque manipolazione della struttura dati viene fatta solo ed esclusivamente attraverso l'uso dell'interfaccia.
- Anche nel caso di strutture dati composite si può utilizzare il concetto di ADT a vantaggio della semplicità e riutilizzabilità del codice.
- Ad esempio, volendo implementare degli ADT relativi agli esempi precedenti potremmo:
 - definire il tipo di dato point con la sua interfaccia: creazione di un punto, modifica di un punto, distruzione di un punto, ...
 - definire il tipo di dato polyline con la sua interfaccia: creazione di una polyline, aggiunta di un punto alla polyline, cancellazione di un punto dalla polyline, distruzione della polyline, ...
 - definire il tipo di dato polylines (lista di polyline) con la sua interfaccia: creazione di una lista di polyline, aggiunta di una polyline alla lista, cancellazione di una polyline dalla lista, ...

Esempio di una rubrica telefonica

ADT number

```
struct phone_number{  
    char label[20];  
    char number[14];  
};
```

```
typedef struct phone_number* phone_number;
```

```
int create_phone_number(phone_number* new_phone_number, char label[], char number[]);
```

```
int get_label(phone_number my_number, char label[]);
```

```
int get_number(phone_number my_number, char number[]);
```

```
int set_label(phone_number my_number, char label[]);
```

```
int set_number(phone_number my_number, char number[]);
```

```
int destroy_phone_number(phone_number* my_phone_number);
```

```
int print_phone_number(phone_number my_phone_number);
```

ADT numbers

```
struct phone_numbers{
    phone_number pn;
    struct phone_numbers* next;
};

typedef struct phone_numbers* phone_numbers;

int create_phone_numbers(phone_numbers* number_list);

int insert_number(phone_numbers* number_list, char new_label[], char new_number[]);

phone_number search_number(phone_numbers number_list, char label_key[]);

int modify_number(phone_numbers number_list, char label_key[], char new_label[], char
new_number[]);

int delete_number(phone_numbers* number_list, char label_key[]);

int destroy_phone_numbers(phone_numbers* number_list);

void print_numbers(phone_numbers number_list);
```

ADT contact

```
struct contact{
    char name[20];
    char surname[20];
    phone_numbers lon;
};

typedef struct contact* contact;

int create_contact(contact* new_contact, char name[], char surname[]);

int get_name(contact my_contact, char name[]);

int get_surname(contact my_contact, char surname[]);

int get_phone_numbers(contact my_contact, phone_numbers** phone_numbers);

int set_name(contact my_contact, char name[]);

int set_surname(contact my_contact, char surname[]);

int destroy_contact(contact* my_contact);

int print_contact(contact my_contact);
```

ADT contacts

```
struct contacts{
    contact c;
    struct contacts* next;
};

typedef struct contacts* contacts;

int create_contacts(contacts* contact_list);

int insert_contact(contacts* contact_list, char new_name[], char new_surname[]);

contact search_contact(contacts contact_list, char surname_key[]);

int modify_contact(contacts contact_list, char surname_key[], char new_name[], char
new_surname[]);

int delete_contact(contacts* contact_list, char surname_key[]);

int destroy_contacts(contacts* contact_list);

void print_contacts(contacts contact_list);
```

Rubrica telefonica

ESERCIZIO 8:

Implementare un programma principale che faccia uso degli ADT appena definiti in modo da fornire all'utente una interfaccia di interazione con la rubrica sotto forma di menu testuale. Tale menù deve offrire sicuramente le seguenti voci:

- inserimento di un contatto con il suo primo numero;
- inserimento di un nuovo numero per un contatto esistente;
- modifica di un numero esistente per un contatto esistente;
- cancellazione di un numero per un contatto esistente;
- cancellazione di un contatto.

Reimplementare l'ADT contacts in modo che:

- la lista di contatti sia ordinata per cognome e a parità di cognome per nome (attenzione all'operazione di modifica che necessita di un cambiamento radicale in quanto all'atto della modifica l'ordinamento va mantenuto);
- siano consentiti duplicati sia su cognome che su coppia cognome-nome.

Reimplementare l'ADT numbers in modo che:

- la lista di numeri sia ordinata per label (attenzione all'operazione di modifica che necessita di un cambiamento radicale in quanto all'atto della modifica l'ordinamento va mantenuto);
- non siano consentiti duplicati su label o su numero.

Implementare sugli ADT numbers e contacts le funzioni `is_empty()` che restituiscono 1 se la lista è vuota e 0 in caso contrario: utilizzare la funzione quando necessario nell'implementazione delle altre operazioni del ADT.

Come realizzare il progetto

Fasi principali

- Concordare con il docente l'argomento e le funzionalità di massima che il programma da realizzare deve fornire.
- Stilare un documento di progetto e sottoporre il documento all'analisi del docente (vedi dettagli su documento di progetto nelle slide successive).
- Una volta concordata la versione definitiva del documento di progetto, implementare il progetto senza utilizzo di file (le strutture dati utilizzate nel progetto devono essere popolate unicamente tramite I/O da terminale).
- Sottoporre il programma ad un debugging accurato tramite l'utilizzo dei tool consigliati nelle slide successive.
- Implementare caricamento e salvataggio delle strutture dati da file rispettivamente all'avvio e alla chiusura del programma (non è necessario salvare su file per ogni operazione di modifica della struttura dati).
- Sottoporre nuovamente il programma ad un debugging accurato tramite l'utilizzo dei tool consigliati nelle slide successive prestando particolare attenzione al fatto che non devono essere presenti:
 - salti dipendenti da variabili non inizializzate;
 - memory leaks;
 - memoria allocata non liberata all'atto della chiusura del programma.
- Consegna del progetto e colloquio informale per dimostrare di essere l'unico autore del programma.

Struttura del documento di progetto

- Intestazione con Nome, Cognome, Matricola e titolo del progetto.
- Descrizione breve in linguaggio naturale del progetto e delle sue funzionalità.
- Descrizione dell'interfaccia fornita all'utente sotto forma di grafo di screenshot in cui si mostra il menu iniziale del programma e i sottomenu raggiungibili da questo grazie all'inserimento di input da tastiera (nota: il menu del programma può essere realizzato in C facilmente anche se non si sono ancora implementate le strutture dati).
- Descrizione degli ADT utilizzati nel programma sotto forma degli header file (definizione del tipo di dato e prototipi delle funzioni dell'interfaccia).
- Descrizione in linguaggio naturale con elenco puntato della corrispondenza tra le operazioni fornite dal programma in forma di menu testuale e le operazioni dell'interfaccia degli ADT coinvolti.
- NOTA: è vietato l'utilizzo di array se non per l'implementazione di stringhe.

Strumenti di debugging

Strumenti di debugging

- Durante l'implementazione del progetto è consigliato l'utilizzo del tool GDB (GNU Debugger) per risolvere bug funzionali (bug sul corretto funzionamento del programma).
- Durante l'implementazione del progetto è obbligatorio l'utilizzo del tool Valgrind per verificare la presenza di:
 - salti dipendenti da variabili non inizializzate;
 - memory leaks;
 - memoria allocata non liberata all'atto della chiusura del programma.
- L'output di Valgrind alla chiusura del programma deve essere simile al seguente:

```
==44475== HEAP SUMMARY:
==44475== in use at exit: 0 bytes in 0 blocks
==44475== total heap usage: 23 allocs, 23 frees, 1,634 bytes allocated
==44475==
==44475== All heap blocks were freed -- no leaks are possible
==44475==
==44475== For lists of detected and suppressed errors, rerun with: -s
==44475== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Gestione dei memory leaks

```
#include <stdio.h>
#include <stdlib.h>
#include "number.h"
#include "numbers.h"
#include "contact.h"
#include "contacts.h"

int main(){
    contacts rubrica;
    create_contacts(&rubrica);

    insert_contact(&rubrica, "Francesco", "Longo");
    insert_contact(&rubrica, "Mario", "Rossi");

    my_contact = search_contact(rubrica, "Longo");
    get_phone_numbers(my_contact, &pointer_to_my_numbers);
    insert_number(pointer_to_my_numbers, "Cellulare", "090");
    insert_number(pointer_to_my_numbers, "Lavoro", "091");

    contact my_contact = search_contact(rubrica, "Rossi");
    phone_numbers* pointer_to_my_numbers;
    get_phone_numbers(my_contact, &pointer_to_my_numbers);
    insert_number(pointer_to_my_numbers, "Casa", "092");
    insert_number(pointer_to_my_numbers, "Casa2", "093");

    print_contacts(rubrica);

    delete_contact(&rubrica, "Longo");
    print_contacts(rubrica);

    destroy_contacts(&rubrica);
}
```

Gestione dei memory leaks (2)

- Ipotizziamo di commentare l'ultima riga (invocazione della funzione `destroy_contacts`)
- L'output di Valgrind in questo caso è:

```
==30686==  
==30686== HEAP SUMMARY:  
==30686==    in use at exit: 164 bytes in 6 blocks  
==30686== total heap usage: 13 allocs, 7 frees, 1,352 bytes allocated  
==30686==  
==30686== LEAK SUMMARY:  
==30686==    definitely lost: 16 bytes in 1 blocks  
==30686==    indirectly lost: 148 bytes in 5 blocks  
==30686==    possibly lost: 0 bytes in 0 blocks  
==30686==    still reachable: 0 bytes in 0 blocks  
==30686==    suppressed: 0 bytes in 0 blocks  
==30686== Rerun with --leak-check=full to see details of leaked memory  
==30686==  
==30686== For lists of detected and suppressed errors, rerun with: -s  
==30686== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

- Esso evidenzia che all'atto della chiusura del programma c'è ancora memoria allocata nell'heap

NOTA: Durante l'implementazione del progetto è necessario assicurarsi che alla chiusura del programma non vi sia memoria allocata nell'heap ovvero che tutte le strutture dati siano state deallocate correttamente.

Gestione dei memory leaks (3)

- Ipotizziamo adesso di ripristinare l'ultima riga del main (invocazione della funzione `destroy_contacts`) ma di commentare la riga che dealloca i contatti all'atto della distruzione della rubrica (invocazione della funzione `destroy_contact` all'interno della funzione `destroy_contacts`)
- L'output di Valgrind in questo caso è:

```
==31769==
==31769== HEAP SUMMARY:
==31769==    in use at exit: 148 bytes in 5 blocks
==31769==   total heap usage: 13 allocs, 8 frees, 1,352 bytes allocated
==31769==
==31769== LEAK SUMMARY:
==31769==    definitely lost: 48 bytes in 1 blocks
==31769==    indirectly lost: 100 bytes in 4 blocks
==31769==    possibly lost: 0 bytes in 0 blocks
==31769==    still reachable: 0 bytes in 0 blocks
==31769==         suppressed: 0 bytes in 0 blocks
==31769== Rerun with --leak-check=full to see details of leaked memory
==31769==
==31769== For lists of detected and suppressed errors, rerun with: -s
==31769== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

- Questi sono dei memory leak più gravi in quanto il programmatore pensa di aver deallocato tutta la memoria nell'heap quando in realtà questo non è vero.

NOTA: E' possibile lanciare valgrind con l'opzione `--leak-check=full` per avere informazioni su dove tale memoria è stata allocata e quindi poterla liberare correttamente. A tal fine è importante compilare tutto con l'opzione `-g` che consente di avere informazione sui numeri di riga nel codice.

Salti dipendenti da variabili non inizializzate

- Consideriamo il seguente main:

```
#include <stdio.h>

int main(){
    int a;

    if(a > 0)
        printf("Maggiore di zero!\n");
    else
        printf("Minore o uguale a zero!\n");
}
```

- Se mandiamo il programma in esecuzione abbiamo il seguente output (che apparentemente sembra corretto):

```
Minore o uguale a zero!
```

- In realtà la variabile `a` non è mai stata inizializzata quindi il flusso logico del programma è compromesso.

Salti dipendenti da variabili non inizializzate (2)

- Se lanciamo il programma tramite Valgrind invece abbiamo il seguente output:

```
==35207== Memcheck, a memory error detector
==35207== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==35207== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==35207== Command: ./a.out
==35207==
==35207== Conditional jump or move depends on uninitialised value(s)
==35207==    at 0x109159: main (prova.c:6)
==35207==
Minore o uguale a zero!
==35207==
==35207== HEAP SUMMARY:
==35207==    in use at exit: 0 bytes in 0 blocks
==35207==   total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==35207==
==35207== All heap blocks were freed -- no leaks are possible
==35207==
==35207== Use --track-origins=yes to see where uninitialised values come from
==35207== For lists of detected and suppressed errors, rerun with: -s
==35207== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

- Ovvero Valgrind ci avverte che c'è una condizione che dipende da variabili non inizializzate.

NOTA: Durante l'implementazione del progetto è necessario assicurarsi che Valgrind non dia questo genere di errori andando ad eseguire il programma su tutti i possibili flussi di esecuzione.

Salti dipendenti da variabili non inizializzate (3)

- Un esempio di questo genere di errore nel programma della rubrica si ottiene non inizializzando i puntatori alla testa delle liste che utilizziamo quindi ad esempio commentando l'invocazione della funzione `create_contacts` nel `main`.
- L'output sarebbe il seguente:

```
==38163== Conditional jump or move depends on uninitialised value(s)
==38163==    at 0x109E91: print_contacts (contacts.c:83)
==38163==    by 0x1092FF: main (main.c:96)
...
==38163== Conditional jump or move depends on uninitialised value(s)
==38163==    at 0x109E91: print_contacts (contacts.c:83)
==38163==    by 0x10931E: main (main.c:100)
...
==38163== Conditional jump or move depends on uninitialised value(s)
==38163==    at 0x109E5A: destroy_contacts (contacts.c:74)
==38163==    by 0x10932A: main (main.c:102)
```

- Nota che l'output del programma lanciato senza Valgrind il più delle volte sembra normale in questo caso: il programma sembra funzionare correttamente perché casualmente le deferenziamenti del puntatore non inizializzato non provocano l'uccisione del processo da parte del sistema operativo.
- Ma tutto ciò è assolutamente casuale e c'è la possibilità che il sistema operativo uccida il processo: esiste un bug pericoloso di cui non mi rendo conto!

Utilizzo di GDB

- L'utilizzo più comune di GDB è tramite l'aggiunta di breakpoint ovvero punti nei quali vogliamo che il nostro programma si fermi per fare delle analisi del suo stato
- Lancio del programma
`gdb -q ./main`

Comandi:

- *run*: fa eseguire il programma fino alla fine o fino al primo breakpoint
- *breakpoint 14*: setta il breakpoint alla riga 14
- *print a*: stampa la variabile a
- *continue*: continua fino alla fine o fino al prossimo breakpoint
- *step*: vai avanti di una istruzione
- *set var c=1*: pone il valore della variabile c a 1
- *print struttura.campo*: se la variabile struttura è una struct posso accedere ai suoi campi con il punto